

Technical Documentation

1. System Overview

This project is a Retrieval-Augmented Generation (RAG) system for answering natural-language questions about the content of a YouTube video, using its transcript. It consists of:

- A **Streamlit frontend** (`app.py`) that collects a YouTube URL and, later, chat questions, communicating with a backend purely over HTTP.
- A **FastAPI backend** (`set_up_file.py`) that owns the RAG pipeline lifecycle: transcript ingestion, chunking, embedding, vector-store persistence, retriever construction, and question answering.
- A **retrieval subsystem** (`Retreivers/`) implementing five selectable retrieval strategies through a single factory function.
- A **reranking module** (`Reranking/reranker.py`) using a cross-encoder, invoked by the evaluation scripts.
- An **evaluation subsystem** (`Exact_retrieval_metrics.py`, `build_ragas_dataset.py`, `Semantic_Meaning_Test.py`) that is entirely decoupled from the live chatbot — it is run manually against labeled datasets to measure retrieval and generation quality.
- A **dataset-authoring helper** (`Building_ready_prompt.py` + `Ingestion/utils.py`) that prepares (but does not itself execute) a prompt for generating labeled evaluation questions from a video's transcript.

2. Architecture

See the Mermaid diagrams in `README.md` Section 4 for the high-level architecture, ingestion pipeline, query pipeline, and evaluation pipeline. This document focuses on module-level and function-level detail.

At a component level, the system has four largely independent execution contexts:

1. **Live serving** — `app.py` (Streamlit) ↔ `set_up_file.py` (FastAPI) ↔ `Ingestion/`, `Pipeline/rag_pipeline.py`, `Retreivers/`.
2. **Dataset authoring** — `Building_ready_prompt.py` ↔ `Ingestion/utils.py`, `Ingestion/youtube_loader.py`, `Ingestion/chunker.py`. Produces a prompt string; the actual dataset JSON is produced by an external LLM call not present in the uploaded code.
3. **Retrieval evaluation** — `Exact_retrieval_metrics.py` ↔ `Ingestion/Vector_DB.py`, `Retreivers/factory.py`, `Reranking/reranker.py`.
4. **Generation/RAG evaluation** — `build_ragas_dataset.py` (produces the dataset) → `Semantic_Meaning_Test.py` (scores it with `ragas`).

3. Directory Structure

Reconstructed from import paths and `sys.path` manipulation found in the source (see README Section 5 for the caveat on inferred folder names).

```
text

project_root/
├─ app.py
├─ set_up_file.py
├─ Building_ready_prompt.py
├─ Ingestion/
│   ├─ utils.py
│   ├─ youtube_loader.py
│   ├─ chunker.py
│   └─ Vector_DB.py
├─ Pipeline/
│   └─ rag_pipeline.py
├─ Retreivers/
│   ├─ factory.py
│   ├─ mmr.py
│   ├─ similarity.py
│   ├─ Build_BM25_retriever.py
│   ├─ hybrid_search_retriever.py
│   ├─ multi_query_retriever.py
│   └─ compression.py
├─ Reranking/
│   └─ reranker.py
└─ Evaluation/ [Needs clarification: name inferred]
    ├─ Exact_retrieval_metrics.py
    ├─ build_ragas_dataset.py
    └─ Semantic_Meaning_Test.py
```

4. Module-by-Module Documentation

File: `app.py`

Responsibility: Streamlit-based chat UI. Presents a URL-submission screen, then a chat screen, and communicates exclusively with the FastAPI backend over HTTP (`requests`). Contains no RAG logic itself.

Imports: `streamlit as st`, `requests`, `time`.

Classes: None.

Functions:

- `prepare_video(youtube_url)`
 - Purpose: Calls the backend's `GET /Youtube_url` endpoint to trigger ingestion for a video.
 - Parameters: `youtube_url` (str).
 - Returns: `(True, data_dict)` on success (`data_dict` contains `video_id`, `status`); `(False, error_message)` on any failure (non-200 status, missing `"status": "ready"`, connection error, timeout, or any other exception).
 - Side effects: makes a GET HTTP request with a 600-second timeout.
 - Called by: the `if start_button:` block in the URL-page section of the script (module-level, not inside a function).
- `ask_question(question)`
 - Purpose: Calls the backend's `POST /ask` endpoint with a question.
 - Parameters: `question` (str).
 - Returns: `(True, answer_text)` on success; `(False, error_message)` otherwise (non-200 status returns raw `response.text`; missing `"answer"` key returns `data.get("error", "Unknown backend error.")`).
 - Side effects: POST HTTP request with a 600-second timeout.
 - Called by: the chat-input handling block in the "CHAT PAGE" section.
- `reset_app()`
 - Purpose: Resets all `st.session_state` fields (`stage`, `video_id`, `youtube_url`, `messages`) back to their initial values.
 - Parameters: none.
 - Returns: None (mutates `st.session_state`).
 - Called by: the "+" + "New Video" button handler.

Control flow: The script is a single top-to-bottom Streamlit script gated by `st.session_state.stage` (`"url"` or `"chat"`). No RAG/LLM calls occur in this file — all such logic is delegated to the FastAPI backend via `prepare_video`/`ask_question`.

File: `set_up_file.py`

Responsibility: FastAPI backend entry point. Owns the `config` dict (LLM, embeddings, reranker instances), the `app` FastAPI object, the two HTTP endpoints, and the video-to-pipeline construction logic.

Imports: `os`, `sys`, `warnings`, `dotenv.load_dotenv`, `joblib.{dump, load}`, `urllib.parse.{urlparse, parse_qs}`, `fastapi.FastAPI`, `pydantic.BaseModel`, `langchain_ollama.OllamaEmbeddings`, `langchain_groq.ChatGroq`, `sentence_transformers.CrossEncoder`, and internal modules `Ingestion.chunker.{chunk_documents, chunk_documents_by_characters}`, `Ingestion.youtube_loader.{load_documents, load_documents_v2}`, `Ingestion.Vector_DB.{creat_vector_DB, load_vector_DB}`, `Pipeline.rag_pipeline.rag_pipeline`, `Retreivers.factory.build_retriever`.

Module-level side effects at import time:

- `warnings.filterwarnings(action="ignore")` — suppresses all warnings globally.
- `load_dotenv()` — loads `.env`.
- `os.environ["GROQ_API_KEY"] = os.getenv("Groq_api_key")` — copies the custom-named env var into the standard name `ChatGroq` expects by default.
- `THIS_DIR`/`PROJECT_ROOT` computed and `PROJECT_ROOT` appended to `sys.path` if not already present, so the `Ingestion.*`, `Pipeline.*`, `Retreivers.*` package imports resolve.
- `DATASETS_DIR`, `CHUNKS_DIR` — hardcoded Windows absolute paths.
- `app = FastAPI()`.
- `rag_pipeline_v1 = None`, `video_id_current = None` — global mutable state.
- `config = {...}` — instantiates `ChatGroq(model_name="openai/gpt-oss-120b", temperature=0.2)` (no explicit `api_key`, relies on the `GROQ_API_KEY` env var set above), `OllamaEmbeddings(model="bge-m3", keep_alive=-1)`, `CrossEncoder(model_name_or_path="cross-encoder/ms-`

`marco-MiniLM-L-6-v2")`). Note: this reranker instance is stored in `(config)` but is **not** referenced anywhere else in this file — reranking is not called from the live pipeline.

Classes:

- `(QuestionRequest(BaseModel))` — Pydantic request body model for `(POST /ask)`, with a single field `(question: str)`.

Functions:

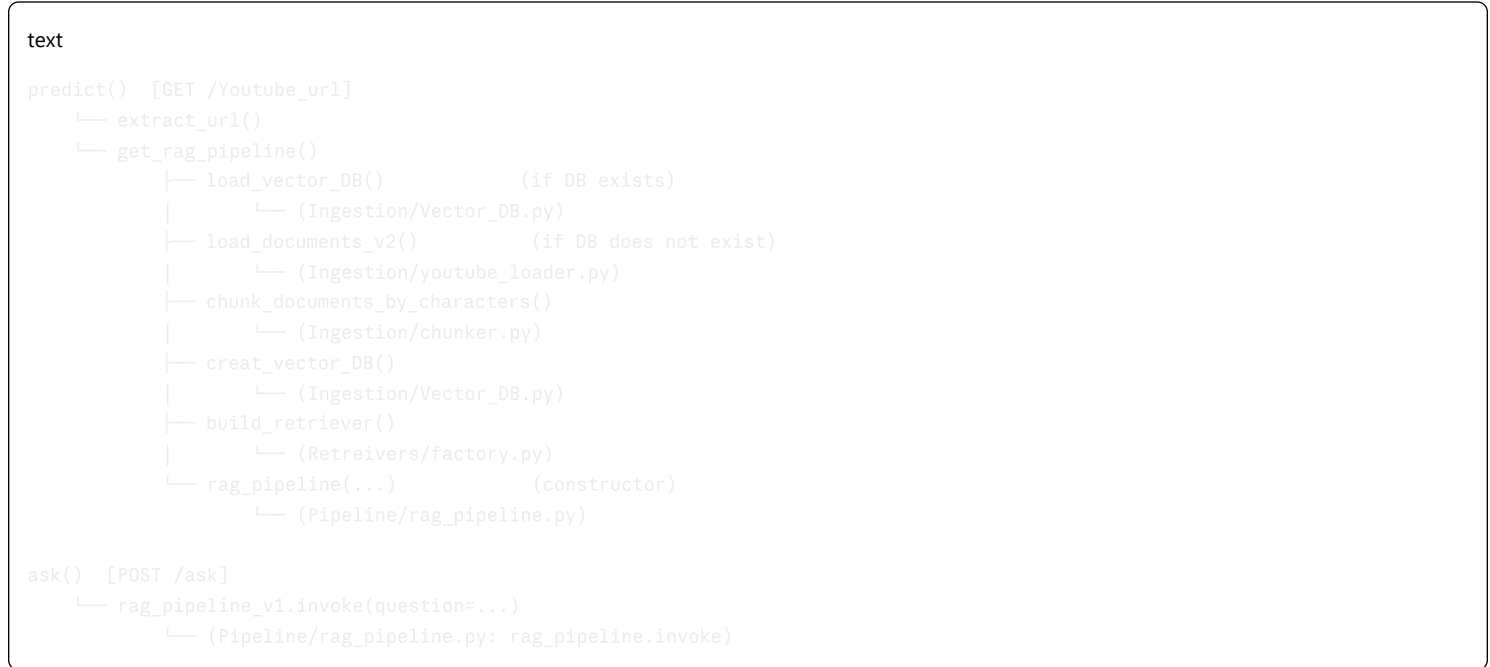
- `(extract_url(youtube_url: str) -> str)`
 - Purpose: Extracts the `(v)` query-parameter (video ID) from a standard `(youtube.com/watch?v=...)` URL.
 - Parameters: `(youtube_url)` (str).
 - Returns: video ID (str).
 - Side effects: none (pure function), but will raise `(KeyError)` if the URL has no `(v)` query parameter (e.g. a `(youtu.be/...)` short link) — **this function does not handle `(youtu.be)` links**, unlike `(youtube_loader.extract_video_id)`, which does.
 - Called by: the `(predict)` endpoint handler.
- `(get_rag_pipeline(video_id: str, youtube_url: str))`
 - Purpose: The central orchestration function — returns a ready-to-use `(rag_pipeline)` instance for a given video, either by loading a previously built vector DB + chunks, or by ingesting the video from scratch.
 - Parameters: `(video_id)` (str), `(youtube_url)` (str).
 - Returns: a `(rag_pipeline)` instance (see `(Pipeline/rag_pipeline.py)`).
 - Side effects: filesystem reads/writes (Chroma persistence, `(joblib)` chunk dump/load), network calls (transcript fetch, embedding calls to Ollama), and object construction (retriever, LLM wrapper objects).
 - Logic:
 - If `(database_path = os.path.join(DATASETS_DIR, f"new_vid_{video_id}"))` exists:
 - `(load_vector_DB(...), load(filename=os.path.join(CHUNKS_DIR, video_id)))` (the previously dumped chunk list),
 - `(build_retriever(..., search_type="mmr_hybrid", k=3, fetch_k=5, lambda_mult=0.5, documents=docs, multi_query=None, compression=True))`,
 - `(rag_pipeline(llm=config["LLM"], retriever=retriever))`.
 - Else:
 - `(load_documents_v2(youtube_url=youtube_url))`,
 - `(chunk_documents_by_characters(documents=docs, chunk_size=1000, chunk_overlap=150))` (the `(SemanticChunker)`-based `(chunk_documents)` call is present in the source but commented out),
 - `(creat_vector_DB(collection_name=f"new_vid_{video_id}", persist_directory=database_path, embedding_function=config["embeddings"], chunks=chunks))`,
 - `(build_retriever(..., search_type="mmr_hybrid", k=3, fetch_k=5, lambda_mult=0.5, documents=chunks, multi_query=None, compression=None))` — note `(compression=None)` here vs. `(compression=True)` in the load-existing branch,
 - `(os.makedirs(CHUNKS_DIR, exist_ok=True), dump(value=chunks, filename=os.path.join(CHUNKS_DIR, video_id)))`,
 - `(rag_pipeline(llm=config["LLM"], retriever=retriever))`.
 - Called by: the `(predict)` endpoint handler (`(GET /Youtube_url)`).

Endpoints:

- `(GET /Youtube_url)` — handler `(predict(url: str))`.
 - Input: query parameter `(url)` (YouTube URL, as a plain string — no Pydantic model).
 - Behavior: sets globals `(video_id_current = extract_url(youtube_url=url))`, `(rag_pipeline_v1 = get_rag_pipeline(video_id=video_id_current, youtube_url=url))`.
 - Output: `({"video_id": video_id_current, "status": "ready"})`.
 - Errors: any exception inside `(get_rag_pipeline)` `(extract_url)` (e.g. `(KeyError)` for a `(youtu.be)` URL, `(RuntimeError)` for a missing transcript) propagates as an unhandled exception — FastAPI will return a `(500 Internal Server Error)` by default; there is no explicit `(try/except)` around this handler.
- `(POST /ask)` — handler `(ask(request: QuestionRequest))`.
 - Input: JSON body validated against `(QuestionRequest)` `({"question": "..."})`.
 - Behavior: if `(rag_pipeline_v1 is None)`, returns `({"error": "Please enter a YouTube URL first."})` (HTTP 200, not a 4xx status). Otherwise calls `(rag_pipeline_v1.invoke(question=request.question))` and returns `({"answer": result.content})`.
 - Output: `({"answer": "<text>")` or `({"error": "<text>")`.

- Errors: no explicit exception handling; an exception inside `.invoke(...)` (e.g. an LLM/network error) propagates as an unhandled 500.

Call graph:



File: `Building_ready_prompt.py`

Responsibility: Standalone helper that, given a YouTube URL, produces a ready-to-send prompt (as a plain string) intended for an external LLM call that will generate a labeled evaluation dataset.

Imports: `Ingestion.utils.{assign_ids, save_chunks, get_context, get_ready_prompt}`, `Ingestion.youtube_loader.load_documents`, `Ingestion.chunker.chunk_documents`.

Functions:

- `build_ready_prompt(youtube_url, chunk_file_path, embeddings)`
 - Purpose: End-to-end helper for preparing an evaluation-dataset-generation prompt for one video.
 - Parameters: `youtube_url` (str), `chunk_file_path` (str — where chunks will be persisted via `joblib`), `embeddings` (an embeddings object, e.g. `OllamaEmbeddings`), passed to the semantic chunker).
 - Returns: `ready_prompt` (str) — the fully formatted `Dataset_Generation_Prompt` text, with the chunked transcript inserted as context, ready to be sent to an LLM (manually or by other code not present in the uploaded files).
 - Side effects: writes a `joblib` file of chunks to `chunk_file_path` (via `save_chunks`).
 - Logic: `load_documents(youtube_url)` (note: **v1**, not `load_documents_v2`) → `chunk_documents(documents, embeddings)` (note: **semantic chunking**, not character-based) → `assign_ids(chunks)` → `save_chunks(chunks, vid_name=chunk_file_path)` → `get_context(chunks_file_path=chunk_file_path)` (re-reads the just-saved file) → `get_ready_prompt(context)`.
 - Called by: nothing else in the uploaded files — this appears to be a manually-invoked utility script/entry point.

Note: This module uses `load_documents` (v1) and semantic `chunk_documents`, which differ from the `load_documents_v2` + `chunk_documents_by_characters` combination used in the live serving path (`set_up_file.py`). These are two independent ingestion configurations used for two different purposes (dataset authoring vs. live serving) — not accidental duplicates, per the code-accuracy rule to distinguish rather than assume duplication.

File: `Ingestion/utils.py`

Responsibility: Chunk-metadata utilities, chunk persistence, the evaluation-dataset-generation prompt template, and helpers to build a context string and a ready prompt from persisted chunks.

Imports: `langchain_community.document_loaders.YouTubeLoader` (imported but not used directly in this file — used via `youtube_loader.py` instead; [Needs clarification] why imported here), `langchain_experimental.text_splitter.SemanticChunker` (also unused directly in this file), `langchain_ollama.{OllamaEmbeddings, OllamaLLM}` (unused directly in this file), `joblib.{load, dump}`, `langchain_core.prompts.PromptTemplate`.

Functions:

- `assign_ids(chunks)` — Purpose: mutate each chunk's metadata in place, setting `chunk.metadata['chunk_id'] = f"chunk_No:{index}"` based on enumeration order. Parameters: `chunks` (list of `LangChain Document`). Returns: `None` (mutates in place). Side effects: mutates the metadata dict of every chunk object. Called by: `Building_ready_prompt.build_ready_prompt`.
- `save_chunks(chunks, vid_name)` — Purpose: persist a chunk list to disk. Parameters: `chunks` (list), `vid_name` (str, the output file path). Returns: `None`. Side effects: writes a file via `joblib.dump`. Called by: `Building_ready_prompt.build_ready_prompt`.
- `get_context(chunks_file_path)` — Purpose: load a persisted chunk list and concatenate each chunk's `chunk_id` and `page_content` into a single newline-delimited context string. Parameters: `chunks_file_path` (str). Returns: `context` (str). Side effects: file read via `joblib.load`. Called by: `Building_ready_prompt.build_ready_prompt`.
- `get_ready_prompt(context)` — Purpose: render `Dataset_Generation_Prompt` with the given context and return the plain text. Parameters: `context` (str). Returns: `Ready_prompt.text.strip()` (str). Side effects: none. Called by: `Building_ready_prompt.build_ready_prompt`.

Prompt template — `Dataset_Generation_Prompt` (`PromptTemplate`, `input_variables=["context"]`): Instructs an LLM to generate exactly 10 diverse evaluation examples using only the provided context, copying chunk IDs verbatim, avoiding yes/no questions, and returning **only** a JSON array of exactly 10 objects each with `question`, `ground_truth_answer`, and `relevant_chunk_ids` — with explicit self-verification instructions (starts with `[`, ends with `]`, exactly 10 objects, all chunk IDs exist verbatim in context, no text outside the array). This template's output format is what the `Eval_dataset*.json` files consumed by the evaluation scripts are expected to conform to.

File: `Ingestion/Vector_DB.py`

Responsibility: Thin wrapper functions around `langchain_chroma.Chroma` for creating and loading a persistent vector store.

Imports: `langchain_chroma.Chroma`.

Functions:

- `creat_vector_DB(collection_name, persist_directory, embedding_function, chunks)` [function name spelled as in source — missing the "e" in "create"] — Purpose: build and persist a new Chroma collection from a list of chunk `Document`s. Parameters: `collection_name` (str), `persist_directory` (str), `embedding_function` (an embeddings object), `chunks` (list of `Document`). Returns: a `Chroma` vector store instance. Side effects: writes to disk at `persist_directory`; computes embeddings for every chunk (via `embedding_function`) as part of `Chroma.from_documents`. Called by: `set_up_file.get_rag_pipeline` (ingestion branch).
 - `load_vector_DB(persist_directory, collection_name, embeddings)` — Purpose: load an existing Chroma collection from disk. Parameters: `persist_directory` (str), `collection_name` (str), `embeddings` (an embeddings object used for future query-time embedding). Returns: a `Chroma` vector store instance. Side effects: reads from disk. Called by: `set_up_file.get_rag_pipeline` (load-existing branch), `Exact_retrieval_metrics.py`, `build_ragas_dataset.py`.
-

File: `Ingestion/youtube_loader.py`

Responsibility: Two independent implementations for turning a YouTube URL into LangChain `Document` object(s), plus a shared video-ID extraction helper.

Imports: `langchain_community.document_loaders.YoutubeLoader`, `youtube_transcript_api.YouTubeTranscriptApi`, `langchain_core.documents.Document`, `urllib.parse.{urlparse, parse_qs}`.

Functions:

- `load_documents(youtube_url)` — Purpose (v1): load a transcript via `langchain_community.document_loaders.YoutubeLoader`, trying `language="en"` then `language="ar"` in a loop, returning as soon as a non-empty document list is loaded. Parameters: `youtube_url` (str). Returns: list of `Document`. Side effects: network call to fetch the transcript (via the underlying `youtube-transcript-api`-based loader), `print()` calls for progress. Raises: `RuntimeError` if neither language succeeds. Called by: `Building_ready_prompt.build_ready_prompt`.
- `extract_video_id(youtube_url)` — Purpose: extract a video ID from either a `youtube.com/watch?v=...` URL or a `youtu.be/<id>` short URL. Parameters: `youtube_url` (str). Returns: video ID (str). Logic: if `"youtu.be"` is in the netloc, returns the stripped path; otherwise returns `parse_qs(parsed_url.query)["v"][0]`. Called by: `load_documents_v2` (in this file).
- `load_documents_v2(youtube_url)` — Purpose (v2, used in production): fetch a transcript using `youtube_transcript_api.YouTubeTranscriptApi` directly (rather than through LangChain's loader), listing all available transcripts, then selecting the first available of `["en", "ar"]` via `transcript_list.find_transcript([language])`, fetching it, joining all caption items' `.text` with spaces, and wrapping the result in a single `Document` with metadata `{"video_id", "language", "is_generated"}`. Parameters: `youtube_url` (str). Returns: list containing exactly one `Document`. Side effects: network call(s) to the transcript API, extensive `print()` diagnostics. Raises: `RuntimeError` if no English or Arabic transcript is found. Called by: `set_up_file.get_rag_pipeline` (ingestion branch) — the comment in `set_up_file.py` explicitly states v2 is used "for better performance".

Difference between `load_documents` and `load_documents_v2`: `load_documents` relies on LangChain's `YoutubeLoader` abstraction and returns whatever document splitting that loader produces (potentially multiple documents); `load_documents_v2` uses `youtube_transcript_api` directly, inspects all available transcripts explicitly (logging their language/generated status), and deliberately concatenates the entire transcript into a **single**

`Document`. These are not duplicates — they are two different loading strategies used in two different parts of the project (dataset authoring vs. live serving, respectively).

File: `Ingestion/chunker.py`

Responsibility: Three interchangeable chunking (text-splitting) strategies for turning `Document`s into smaller `Document` chunks.

Imports: `langchain_experimental.text_splitter.SemanticChunker`, `langchain_text_splitters.TokenTextSplitter`, `langchain_text_splitters.RecursiveCharacterTextSplitter`.

Functions:

- `chunk_documents(documents, embeddings, breakpoint_threshold_type, breakpoint_threshold_amount)` — Purpose: semantic chunking using embedding-distance breakpoints. Parameters: `documents` (list of `Document`), `embeddings` (embeddings object), `breakpoint_threshold_type` (str, e.g. "standard_deviation"), `breakpoint_threshold_amount` (numeric). Returns: list of chunked `Document`. Called by: `Building_ready_prompt.build_ready_prompt`; present but commented out in `set_up_file.py`.
- `chunk_documents_by_tokens(documents, chunk_size=500, chunk_overlap=50)` — Purpose: token-count-based chunking via `TokenTextSplitter`. Parameters as named, with defaults. Returns: list of chunked `Document`. Called by: no other uploaded file — an available-but-unused utility. `[Needs clarification]`.
- `chunk_documents_by_characters(documents, chunk_size=500, chunk_overlap=50)` — Purpose: fixed-size character chunking with Arabic-aware separators (`(["\n\n", "\n", ":", "!", ".", ":", " ", ""])`) via `RecursiveCharacterTextSplitter`. Parameters as named, with defaults (called with `chunk_size=1000, chunk_overlap=150`) in `set_up_file.py`. Returns: list of chunked `Document`. Called by: `set_up_file.get_rag_pipeline` (ingestion branch) — the production chunking method.

File: `Pipeline/rag_pipeline.py`

Responsibility: Defines the `rag_pipeline` class, which executes a single retrieval → prompt → generation turn given an already-constructed `llm` and `retriever`.

Imports: `json`, `os`, `sys`, `time`, `langchain_core.prompts.PromptTemplate`, `langchain_ollama.{OllamaLLM, OllamaEmbeddings}`, `Retrievers.mmr.build_mmr_retrievers`, `Ingestion.Vector_DB.load_vector_DB`, `Retrievers.factory.build_retriever`. Also computes `THIS_DIR/PROJECT_ROOT` and appends `PROJECT_ROOT` to `sys.path`, same pattern as `set_up_file.py`.

Module-level state:

- `config = {'llm': OllamaLLM(model="qwen3:1.7b"), 'embeddings': OllamaEmbeddings(model="bge-m3")}` — instantiated at import time but **never referenced by the** `rag_pipeline` class, whose constructor takes `llm`/`retriever` as explicit arguments instead. This appears to be dead/leftover code; it does cause a local Ollama model load attempt (`qwen3:1.7b`) at import time regardless of whether it is used. `[Needs clarification]`.

Classes:

- `rag_pipeline`
 - `__init__(self, llm, retriever)` — stores `self.llm = llm`, `self.retriever = retriever`.
 - `invoke(self, question)`
 - Purpose: run one full question-answering turn.
 - Parameters: `question` (str).
 - Returns: the LLM's raw response object (in the `set_up_file.py` usage, `.content` is read from it afterward, implying an object like `ChatGroq`'s `AIMessage`).
 - Side effects: network/compute calls to the retriever and the LLM; `print()` output of three latency measurements.
 - Logic:

```
text

docs = self.retriever.invoke(input=question)          # timed: retrieval_start/end
context = "\n\n".join(doc.page_content for doc in docs)
prompt = PromptTemplate(template="...", input_variables=["question", "context"])
ready_prompt = prompt.invoke(input={'question': question, 'context': context}) # timed: build_prompt
answer = self.llm.invoke(input=ready_prompt)          # timed: generation
print(...) # three latency lines
return answer
```

- Called by: `set_up_file.py`'s `ask()` endpoint handler (`'rag_pipeline_v1.invoke(question=request.question)'`).

Prompt template (defined inline inside `invoke`): Instructs the LLM to answer only from the given context, to answer in the same language as the question (English or Arabic), to keep technical/domain terminology in English where that is the natural form rather than translating or duplicating it, and to state that the information is unavailable if it cannot be found in the context. The template is rebuilt on every call to `invoke` rather than being defined once at module or class scope. `[Needs clarification]`: this is a minor inefficiency (re-constructing an identical `PromptTemplate` object every call) rather than a correctness issue.

File: `Exact_retrieval_metrics.py`

Responsibility: Standalone evaluation script measuring exact-match retrieval quality (Recall/Precision/F1) and latency against labeled `Eval_dataset*.json` files, for three hardcoded videos.

Imports: `(sys, os, joblib.load, dotenv.load_dotenv, time, json, langchain_core.prompts.PromptTemplate)` (imported but not used in this file — `[Needs clarification]`), `(langchain_ollama.{OllamaLLM, OllamaEmbeddings}, sentence_transformers.CrossEncoder, langchain_groq.ChatGroq, Ingestion.Vector_DB.load_vector_DB, Retrievers.factory.build_retriever, Reranking.reranker.Rerank)`.

Module-level config: `(ChatGroq(model_name='openai/gpt-oss-20b', api_key=os.getenv(key='Groq_api_key'), temperature=0.2))` (note: `openai/gpt-oss-20b`, a different model from the `openai/gpt-oss-120b` used in `set_up_file.py`), `(OllamaEmbeddings(model='bge-m3'))`, `(CrossEncoder(model_name_or_path='cross-encoder/ms-marco-MiniLM-L-6-v2'))`.

Functions:

- `calc_recall_precesion_f1_score(relevant_chunks, retrieved_chunks)` [name spelled as in source] — Purpose: compute Recall, Precision, F1 (as percentages) from two lists of chunk IDs. Parameters: `(relevant_chunks)` (list of str), `(retrieved_chunks)` (list of str). Returns: `((recall, precesion, f1_score))` tuple of floats; all `0` if the intersection is empty. Side effects: `(print())`s each metric. Called by: `Test_Retrieval_process`.
- `Test_Retrieval_process(Evaluation_dataset_file_path, vector_DB, documents=None)`
 - Purpose: run the full retrieval-evaluation loop over one labeled dataset.
 - Parameters: `(Evaluation_dataset_file_path)` (str, path to a JSON file of `{question, ground_truth_answer, relevant_chunk_ids}` objects), `(vector_DB)` (a loaded `Chroma` instance), `(documents)` (list of chunk `Document`, required for BM25 inside `build_retriever`).
 - Returns: `(None)` (prints per-question and averaged Recall/Precision/F1/latency).
 - Side effects: builds a new retriever per question, network calls to the vector DB and reranker, extensive `(print())` output.
 - Logic per item: builds `(build_retriever(vector_db=vector_DB, llm=config['LLM'], search_type='mmr_hybrid', k=len(item['relevant_chunk_ids']), fetch_k=len(item['relevant_chunk_ids'])*2, lambda_mult=0.5, documents=documents, multi_query=None, compression=None))` → `(retriever.invoke(input=item['question']))` → `(Rerank(reranker_model=config['reranker'], query=item['question'], docs=result, Top_k=len(item['relevant_chunk_ids'])))` → extracts `(retrieved_chunks)` from `(reranked_docs[i].metadata['chunk_id'])` → `(calc_recall_precesion_f1_score(...))`.
 - Called by: the module-level loop over `(combinations)` at the bottom of the file.

Module-level execution (runs on import/script execution, not gated behind `if __name__ == "__main__":`): loads three `Chroma` DBs (`(DB_VID1, DB_VID2, DB_VID3)`) from hardcoded paths, then iterates `(combinations)` (a list of `(eval_dataset_path, vector_db, chunks_path)` tuples for three videos) calling `(Test_Retrieval_process(..., documents=load(item[2])))` for each.

Call graph:

```
text
module execution
  ↳ load_vector_DB() x3
  ↳ for each of 3 combinations:
    ↳ Test_Retrieval_process()
      ↳ build_retriever() [per question]
        ↳ (Retrievers/factory.py)
      ↳ retriever.invoke()
      ↳ Rerank()
        ↳ (Reranking/reranker.py)
      ↳ calc_recall_precesion_f1_score()
```

File: `Semantic_Meaning_Test.py`

Responsibility: Scores a pre-built Ragas-format dataset (`(Ragas_Eval_dataset*.json)`) using four `(ragas)` metrics, via a Groq-hosted evaluator LLM and an Ollama-hosted evaluator embedding model.

Imports: `(json, os, openai.AsyncOpenAI, asyncio, dotenv.load_dotenv, ragas.SingleTurnSample, ragas.llms.llm_factory, ragas.embeddings.base.embedding_factory, ragas.metrics.collections.{ContextRecall, ContextPrecision, AnswerRelevancy, Faithfulness})`.

Module-level state:

- `groq_client = AsyncOpenAI(api_key=os.getenv(key="Groq_api_key"), base_url="https://api.groq.com/openai/v1", timeout=300)`.
- `ollam_client = AsyncOpenAI(api_key="ollama", base_url="http://localhost:11434/v1", timeout=300)` [note: variable name spelled `ollam_client` in source].
- `llm_evaluator = llm_factory(model="openai/gpt-oss-20b", provider="openai", client=groq_client, max_tokens=4096)`.
- `embeddings = embedding_factory(model="bge-m3", client=ollam_client, provider="openai")`.
- `context_precision = ContextPrecision(llm=llm_evaluator)`, `context_recall = ContextRecall(llm=llm_evaluator)`, `answer_relevancy = AnswerRelevancy(llm=llm_evaluator, embeddings=embeddings)`, `faithfulness = Faithfulness(llm=llm_evaluator)` [note: variable name spelled `faithfulness` in source].

Functions (all `async`):

- `Calc_faithfulness(sample)` — calls `faithfulness.ascore(user_input, response, retrieved_contexts)`, returns `{'faithfulness_score': ...value}`. Not called in `main()` as written.
- `Calc_answer_relevancy(sample)` — calls `answer_relevancy.ascore(user_input, response)`, returns `{'answer_relevancy_score': ...value}`. Not called in `main()` as written.
- `Calc_context_precision(sample)` — calls `context_precision.ascore(user_input, reference, retrieved_contexts)`, returns `{'context_precision_score': ...value}`. **This is the only one called from `main()`.**
- `Calc_context_recall(sample)` — calls `context_recall.ascore(user_input, retrieved_contexts, reference)`, returns `{'context_recall_score': ...value}`. Not called in `main()` as written.
- `Evaluate(sample)` — runs all four `.ascore(...)` calls concurrently via `asyncio.gather`, returns a dict with all four scores. Defined but not called from `main()` as uploaded. [Needs clarification]: whether this omission from `main()` is intentional (e.g. work in progress) or an oversight.
- `main()` — opens `files[2]` (i.e., only the **third** of three listed dataset files, `Ragas_Eval_dataset3.json`); the `for file in files:` loop that would process all three is present in the source but commented out), loads it as JSON, builds a `SingleTurnSample` per item (`user_input`, `retrieved_contexts`, `retrieved_context_ids`, `reference_context_ids`, `response`, `reference`) — populated from the corresponding `Ragas_Eval_dataset` fields), calls `await Calc_context_precision(sample=sample)` for each, and prints the per-question result.

Module-level execution: `asyncio.run(main=main())` runs at import/script execution time (not gated behind `if __name__ == "__main__":`).

File: `build_ragas_dataset.py`

Responsibility: Standalone script that runs the retrieval + reranking + generation pipeline over each labeled question in an `Eval_dataset*.json` file, and writes an augmented `Ragas_Eval_dataset*.json` file suitable for `Semantic_Meaning_Test.py`.

Imports: `sys`, `os`, `json`, `joblib.{load, dump}`, `dotenv.load_dotenv`, `langchain_core.prompts.PromptTemplate`, `langchain_ollama`, `{OllamaEmbeddings, OllamaLLM}`, `Ingestion.Vector_DB.load_vector_DB`, `Retrievers.factory.build_retriever`, `Reranking.reranker.Rerank`, `sentence_transformers.CrossEncoder`, `langchain_groq.ChatGroq`.

Module-level config: `ChatGroq(model_name="openai/gpt-oss-120b", api_key=os.getenv(key="Groq_api_key"), temperature=0.2)` (note: **120b** model here, matching production, unlike `Exact_retrieval_metrics.py`'s 20b), `OllamaEmbeddings(model='bge-m3')`, `CrossEncoder(model_name_or_path='cross-encoder/ms-marco-MiniLM-L-6-v2')`.

Functions:

- `calc_recall_precesion_f1_score(relevant_chunks, retrieved_chunks)` — identical implementation to the same-named function in `Exact_retrieval_metrics.py` (duplicated code across the two files rather than shared).
- `build_retrieved_generated_dataset(Evaluation_dataset_file_path, vector_DB, output_file, documents=None)`
 - Purpose: for each labeled question, retrieve, rerank, generate an answer, and record the results back into the dataset item.
 - Parameters: `Evaluation_dataset_file_path` (str), `vector_DB` (Chroma instance), `output_file` (str, output path), `documents` (list of chunk `Document`).
 - Returns: `None` (writes `output_file` as JSON; prints averaged metrics).
 - Logic per item: `build_retriever(vector_db=vector_DB, llm=config['llm'], search_type='mmr_hybrid', k=len(item['relevant_chunk_ids']), fetch_k=len(item['relevant_chunk_ids'])*2, lambda_mult=0.5, documents=documents, multi_query=None, compression=None)` → `retriever.invoke(input=item['question'])` → `Rerank(reranker_model=config['reranker'], query=item['question'], docs=result, Top_k=len(item['relevant_chunk_ids']))` → builds a local `PromptTemplate` (context + question) → `config['llm'].invoke(input=ready_prompt)` → records `item['response']`, `item['retrieved_contexts']`, `item['retrieved_chunk_ids']` → `calc_recall_precesion_f1_score(...)` for reporting.
 - Side effects: writes the augmented dataset to `output_file` via `json.dump(..., ensure_ascii=False, indent=4)`.
 - Called by: the module-level loop over `combinations` at the bottom of the file.

Module-level execution: loads three (Chroma) DBs, then loops over (combinations) (a list of ((eval_dataset_path, vector_db, output_dataset_path, chunks_path) tuples for three videos), calling (build_retrieved_generated_dataset(..., documents=load(item[3]))).

File: (Reranking/reranker.py)

Responsibility: Cross-encoder reranking of a candidate document list against a query.

Imports: none listed explicitly (relies on the (reranker_model) object passed in, e.g. a (sentence_transformers.CrossEncoder) instance).

Functions:

- (Rerank(reranker_model, query, docs, Top_k))
 - Purpose: score every retrieved document against the query with a cross-encoder and return the top (Top_k).
 - Parameters: (reranker_model) (an object exposing (.predict(inputs=...)), e.g. (CrossEncoder)), (query) (str), (docs) (list of (Document)), (Top_k) (int).
 - Returns: list of up to (Top_k) (Document) objects, sorted by descending cross-encoder score.
 - Logic: builds ((query, doc.page_content) pairs for every doc → (scores = reranker_model.predict(inputs=pairs) → (sorted(zip(scores, docs), reverse=True) → takes the (Document) from the first (Top_k) (score, doc) tuples.
 - Called by: (Exact_retrieval_metrics.Test_Retrieval_process), (build_ragas_dataset.build_retrieved_generated_dataset). **Not called from the live (set_up_file.py) chatbot path.**
 - ([Needs clarification]): (reranker_model.predict(inputs=pairs)) uses a keyword argument (inputs=); whether the (CrossEncoder.predict) API in the installed (sentence-transformers) version accepts this exact keyword name could not be verified from the uploaded files alone.
-

File: (Retreivers/factory.py)

Responsibility: Single entry point ((build_retriever)) that constructs a retriever object based on a (search_type) string and optional wrapping flags.

Imports: (Retreivers.mmr.build_mmr_retrievers), (Retreivers.similarity.build_similarity_retriever), (Retreivers.multi_query_retriever.build_multi_query_retriever), (Retreivers.compression.build_compression_retriever), (Retreivers.hybrid_search_retriever.build_ensemble_retriever), (Retreivers.Build_BM25_retriever.build_BM25Retriever).

Functions:

- (build_retriever(vector_db, llm, search_type, k, fetch_k, lambda_mult, documents=None, multi_query=None, compression=None))
 - Purpose: retriever factory/composition root.
 - Parameters: (vector_db) (Chroma instance), (llm) (an LLM object, used only if (multi_query) or (compression) is truthy), (search_type) ("similarity" | "mmr" | "similarity_hybrid" | "mmr_hybrid"), (k)/(fetch_k)/(lambda_mult) (retrieval sizing/diversity params, passed through to the relevant sub-builder), (documents) (chunk list, required for the two (*_hybrid) search types, since BM25 needs it), (multi_query) (truthy → wrap in (MultiQueryRetriever)), (compression) (truthy → wrap in (ContextualCompressionRetriever)).
 - Returns: a retriever object (exact class depends on the combination of flags — could be a plain LangChain (VectorStoreRetriever), an (EnsembleRetriever), a (MultiQueryRetriever), a (ContextualCompressionRetriever), or nested combinations of these).
 - Raises: (ValueError(f"No search type called {search_type}")) for an unrecognized (search_type).
 - Logic:

```
text

if search_type == "similarity": retriever = build_similarity_retriever(vector_db, k)
elif search_type == "mmr": retriever = build_mmr_retrievers(vector_db, k, fetch_k, lambda_mult)
elif search_type == "similarity_hybrid":
    retriever = build_ensemble_retriever(build_similarity_retriever(...), build_BM25Retriever(documents, k))
elif search_type == "mmr_hybrid":
    retriever = build_ensemble_retriever(build_mmr_retrievers(...), build_BM25Retriever(documents, k))
else: raise ValueError(...)

if multi_query: retriever = build_multi_query_retriever(llm, retriever)
if compression: retriever = build_compression_retriever(llm, retriever)
return retriever
```

- Called by: (set_up_file.get_rag_pipeline), (Exact_retrieval_metrics.Test_Retrieval_process), (build_ragas_dataset.build_retrieved_generated_dataset).

Call graph:

text

```
build_retriever()
├─ build_similarity_retriever()      [search_type == "similarity" or half of "similarity_hybrid"]
├─ build_mmr_retrievers()           [search_type == "mmr" or half of "mmr_hybrid"]
├─ build_BM25Retriever()            [either *_hybrid search_type]
├─ build_ensemble_retriever()       [either *_hybrid search_type]
├─ build_multi_query_retriever()    [if multi_query]
└─ build_compression_retriever()    [if compression]
```

File: `Retreivers/mmr.py`

Function: `build_mmr_retrievers(vector_db, k, fetch_k, lambda_mult)` — calls `vector_db.as_retriever(search_type="mmr", search_kwargs={"k": k, "fetch_k": fetch_k, "lambda_mult": 0.5})`. **Bug/behavior note:** the `lambda_mult` parameter passed into the function is never used inside the function body — `0.5` is hardcoded directly in `search_kwargs` instead. Every current caller happens to pass `lambda_mult=0.5` anyway, so this has no observable effect with today's call sites, but the parameter is non-functional. Called by: `Retreivers/factory.py` (both `"mmr"` and `"mmr_hybrid"` branches).

File: `Retreivers/similarity.py`

Function: `build_similarity_retriever(vector_db, k)` — calls `vector_db.as_retriever(search_type="similarity", search_kwargs={"k": k})`. Called by: `Retreivers/factory.py` (`"similarity"` and `"similarity_hybrid"` branches).

File: `Retreivers/Build_BM25_retriever.py`

Function: `build_BM25Retriever(documents, k)` — calls `BM25Retriever.from_documents(documents=documents, k=k)`. Requires the actual `Document` objects (not just the vector store) since BM25 is a lexical index built over document text. Called by: `Retreivers/factory.py` (both `*_hybrid` branches).

File: `Retreivers/hybrid_search_retriever.py`

Function: `build_ensemble_retriever(retriever1, retriever2)` — returns `EnsembleRetriever(retrievers=[retriever1, retriever2], weights=[0.5, 0.5])` from `langchain_classic.retrievers`. Equal-weight rank fusion of two retrievers. Called by: `Retreivers/factory.py` (both `*_hybrid` branches).

File: `Retreivers/multi_query_retriever.py`

Function: `build_multi_query_retriever(llm, retriever)` — returns `MultiQueryRetriever.from_llm(llm=llm, retriever=retriever)` from `langchain_classic.retrievers.multi_query`. Called by: `Retreivers/factory.py`, gated behind `if multi_query:` — never triggered by any current call site (`multi_query=None` everywhere).

File: `Retreivers/compression.py`

Function: `build_compression_retriever(llm, base_retriever)` — builds `LLMChainExtractor.from_llm(llm=llm)` as `base_compressor`, returns `ContextualCompressionRetriever(base_compressor=base_compressor, base_retriever=base_retriever)` from `langchain_classic.retrievers`. Called by: `Retreivers/factory.py`, gated behind `if compression:` — triggered only in `set_up_file.get_rag_pipeline`'s load-existing-DB branch (`compression=True`); not triggered in the first-time-ingestion branch or either evaluation script (all pass `compression=None`).

5. End-to-End Execution Flow

Live serving flow (first-time video):

1. User submits a YouTube URL in Streamlit (`app.py`).
2. `prepare_video()` issues `GET /Youtube_url?url=...` to the FastAPI backend.
3. `set_up_file.predict()` calls `extract_url()` to get the video ID, then `get_rag_pipeline()`.
4. `get_rag_pipeline()` finds no existing database directory, so it calls `load_documents_v2()` (fetch transcript) → `chunk_documents_by_characters()` (chunk) → `creat_vector_DB()` (embed + persist to Chroma) → `dump()` (persist chunks via joblib) → `build_retriever(search_type="mmr_hybrid", compression=None)` → constructs a `rag_pipeline` instance, stored in the global `rag_pipeline_v1`.
5. Backend returns `{"video_id": ..., "status": "ready"}`; Streamlit transitions to the chat screen.
6. User asks a question; `ask_question()` issues `POST /ask`.
7. `set_up_file.ask()` calls `rag_pipeline_v1.invoke(question=...)`, which retrieves via the hybrid (MMR + BM25) retriever, assembles context, builds the prompt, calls `ChatGroq`, and returns the answer object; the endpoint extracts `.content` and returns `{"answer": ...}`.
8. Streamlit displays the answer in the chat.

Live serving flow (previously-processed video): identical, except step 4 takes the `load_vector_DB()` + `load()` (chunks) branch and builds the retriever with `compression=True`.

Evaluation flow (retrieval quality): run `Exact_retrieval_metrics.py` directly; for each of 3 hardcoded videos, iterate each labeled question, build a per-question-sized `mmr_hybrid` retriever, retrieve, rerank with the cross-encoder, compute Recall/Precision/F1 against `relevant_chunk_ids`, and print per-question and averaged results.

Evaluation flow (generation quality): first run `build_ragas_dataset.py` to produce `Ragas_Eval_dataset*.json` (adds `response`, `retrieved_contexts`, `retrieved_chunk_ids` per item by actually running retrieval + reranking + generation), then run `Semantic_Meaning_Test.py` to score `Ragas_Eval_dataset3.json` (only the third file, and only Context Precision, as currently written in `main()`) with `ragas` metrics.

Dataset-authoring flow (manual step, upstream of both evaluation flows): run `Building_ready_prompt.build_ready_prompt(youtube_url, chunk_file_path, embeddings)` to ingest+chunk a video (using `load_documents` v1 and semantic `chunk_documents`) and produce a ready-made LLM prompt string; this string must then be sent to an LLM by some means not present in the uploaded code, and the resulting JSON saved as an `Eval_dataset*.json` file before the evaluation scripts can run against it.

6. Data Flow

```
text

Raw YouTube data (captions via youtube_transcript_api or YoutubeLoader)
→ Documents (LangChain Document objects, 1 per video in load_documents_v2)
→ Chunks (RecursiveCharacterTextSplitter in production; SemanticChunker in dataset-authoring flow)
→ Embeddings (OllamaEmbeddings "bge-m3", computed inside Chroma.from_documents)
→ Vector DB (Chroma, persisted per video_id)
→ Retrieved Documents (hybrid MMR + BM25 via EnsembleRetriever, optionally wrapped in
    ContextualCompressionRetriever when compression=True – live path only reaches this point)
→ Reranked Documents (cross-encoder Rerank() – evaluation scripts only, not the live /ask path)
→ Context (newline/double-newline joined doc.page_content)
→ Prompt (PromptTemplate combining context + question)
→ LLM (ChatGroq: openai/gpt-oss-120b in production, openai/gpt-oss-20b in the exact-metrics evaluator config)
→ Answer (LLM response object; .content read where consumed)
```

7. Retrieval Internals

See Section 4's entries for `Retreivers/factory.py`, `mmr.py`, `similarity.py`, `Build_BM25_retriever.py`, `hybrid_search_retriever.py`, `multi_query_retriever.py`, and `compression.py` above for full source-level detail on each retriever implementation, and README Section 14 for the conceptual explanation of each strategy.

8. Reranking Internals

See Section 4's `Reranking/reranker.py` entry above. In summary: `Rerank()` builds `(query, page_content)` pairs for every candidate document, scores them in a single batched `reranker_model.predict(inputs=pairs)` call, sorts descending by score, and slices the top `Top_k`. It is a pure, synchronous function with no persistence or side effects beyond the model inference call itself.

9. Evaluation Internals

See Section 4's entries for `Exact_retrieval_metrics.py`, `build_ragas_dataset.py`, and `Semantic_Meaning_Test.py` above for full detail. In summary:

- **Exact metrics** operate purely on chunk-ID set overlap (Recall/Precision/F1), independent of any LLM judgment.
- **Ragas metrics** use an LLM (`openai/gpt-oss-20b` via Groq) and, for `AnswerRelevancy`, an embedding model (`bge-m3`) via Ollama's OpenAI-compatible endpoint) as automated judges of semantic quality — these are LLM-as-judge metrics, distinct from the exact chunk-ID matching used by the retrieval-only evaluation script.
- Both evaluation flows depend on a labeled `Eval_dataset*.json` whose creation (the actual LLM call that produces the JSON) is external to the uploaded codebase.

10. Configuration Reference

Parameter	Location	Meaning	Example Value(s) Found in Source
LLM_model_name	set_up_file.py config, build_ragas_dataset.py config	Production generation model.	"openai/gpt-oss-120b"
LLM_model_name (evaluation)	Exact_retrieval_metrics.py config, Semantic_Meaning_Test.py llm_evaluator	Evaluation-time model (smaller than production).	"openai/gpt-oss-20b"
LLM_temperature	set_up_file.py, Exact_retrieval_metrics.py, build_ragas_dataset.py config	Sampling temperature for ChatGroq.	0.2 (consistent everywhere it's set)
embeddings_model	All config dicts using OllamaEmbeddings	Embedding model name served by Ollama.	"bge-m3"
embeddings_keep_alive	set_up_file.py config["embeddings"]	Keeps the Ollama model resident in memory indefinitely.	-1
reranker_model_name_or_path	set_up_file.py, Exact_retrieval_metrics.py, build_ragas_dataset.py config	Cross-encoder checkpoint.	"cross-encoder/ms-marco-MiniLM-L-6-v2"
search_type	passed to build_retriever(...)	Selects retrieval strategy.	"mmr_hybrid" in every call site found
k	build_retriever(...) → sub-builders	Number of documents finally returned by the dense/BM25 retrievers.	3 in set_up_file.py; len(relevant_chunk_ids) in evaluation scripts
fetch_k	build_retriever(...) → build_mmr_retrievers	Candidate pool size before MMR selection.	5 in set_up_file.py; len(relevant_chunk_ids) * 2 in evaluation scripts
lambda_mult	build_retriever(...) → build_mmr_retrievers	Intended relevance/diversity trade-off for MMR — not actually applied (hardcoded to 0.5 inside build_mmr_retrievers, ignoring this argument).	0.5 (passed everywhere, matching the hardcoded value, so the bug is currently unobservable)
multi_query	build_retriever(...)	Enables MultiQueryRetriever wrapping when truthy.	None at every call site — never enabled
compression	build_retriever(...)	Enables ContextualCompressionRetriever wrapping when truthy.	True only in set_up_file.py's load-existing-DB branch; None elsewhere
chunk_size / chunk_overlap (character chunker)	chunk_documents_by_characters(...) call in set_up_file.py	Chunk length and overlap for RecursiveCharacterTextSplitter.	chunk_size=1000, chunk_overlap=150
chunk_size / chunk_overlap (character chunker defaults)	Ingestion/chunker.py function signature	Defaults if not overridden.	chunk_size=500, chunk_overlap=50
breakpoint_threshold_type / breakpoint_threshold_amount (semantic chunker)	commented-out call in set_up_file.py	SemanticChunker breakpoint configuration.	"standard_deviation", 1
DATASETS_DIR	set_up_file.py	Root directory for per-video Chroma persistence.	"D:/Youtube rag system/Datasets"
CHUNKS_DIR	set_up_file.py	Root directory for joblib-dumped chunk lists.	"D:/Youtube rag system/Temp_Chunks"
Top_k	Rerank(...) calls	Number of documents kept after reranking.	len(item['relevant_chunk_ids']) (evaluation scripts only)

Parameter	Location	Meaning	Example Value(s) Found in Source
<code>FASTAPI_URL</code>	<code>app.py</code>	Base URL the Streamlit frontend uses to reach the backend.	<code>"http://127.0.0.1:8000"</code>
<code>languages</code>	<code>youtube_loader.load_documents</code>	Ordered list of transcript languages attempted.	<code>["en", "ar"]</code>
<code>["en", "ar"]</code> (v2)	<code>youtube_loader.load_documents_v2</code>	Same language preference, applied via <code>transcript_list.find_transcript([language])</code> .	<code>["en", "ar"]</code>

11. API / Interface Reference

The project exposes one HTTP API surface: the FastAPI backend defined in `set_up_file.py`.

GET /Youtube_url

- **Handler:** `predict(url: str)`.
- **Input:** query string parameter `url` (str) — a YouTube video URL.
- **Output (200):** `{"video_id": "<str>", "status": "ready"}`.
- **Errors:** No explicit error handling in the handler. Exceptions from `extract_url()` (e.g. `KeyError`) if the URL has no `v=` query param — notably, this function does **not** support `youtu.be` short links, unlike `youtube_loader.extract_video_id`) or from `get_rag_pipeline()` (e.g. `RuntimeError` for a missing transcript, network errors from Ollama/Groq/Chroma) propagate as unhandled exceptions, which FastAPI will turn into a `500 Internal Server Error` with a generic error body.

POST /ask

- **Handler:** `ask(request: QuestionRequest)`.
- **Input:** JSON body `{"question": "<str>"}`, validated by Pydantic.
- **Output (200):**
 - Success: `{"answer": "<str>"}`.
 - No video prepared yet: `{"error": "Please enter a YouTube URL first."}` (still HTTP 200, not a 4xx).
- **Errors:** No explicit `try/except`; exceptions inside `rag_pipeline_v1.invoke(...)` (retriever or LLM failures) propagate as unhandled exceptions → FastAPI default `500`.

No other endpoints, authentication mechanisms, or API versioning were found in the uploaded files.

12. Testing Strategy

No unit-test framework (`pytest`, `unittest`, etc.) or test files were present in the uploaded files. What exists instead is a set of **evaluation scripts** that function as integration-level quality checks rather than automated tests:

- **Retrieval tests:** `Exact_retrieval_metrics.py` — deterministic Recall/Precision/F1 scoring against hand-labeled chunk IDs, run manually as a script, printing results rather than asserting pass/fail thresholds.
- **RAG (generation) tests:** `Semantic_Meaning_Test.py` — LLM-judged Ragas metrics, likewise run manually and printed rather than asserted.
- **Test/evaluation datasets:** `Eval_dataset{1,2,3}.json` (labeled questions + ground truth + relevant chunk IDs — content not included in the uploaded files, only the schema/consumption code) and `Ragas_Eval_dataset{1,2,3}.json` (augmented versions produced by `build_ragas_dataset.py`).
- **Metrics covered:** Recall, Precision, F1, retrieval(+rerank) latency (exact metrics); Context Precision, Context Recall, Answer Relevancy, Faithfulness (Ragas metrics — though as written, `Semantic_Meaning_Test.py`'s `main()` only exercises Context Precision on one of the three dataset files).

There is no CI configuration, no test runner invocation, and no assertions — these scripts are diagnostic/reporting tools rather than a pass/fail test suite.

13. Production Readiness Analysis

Already Implemented

- Persistent, per-video vector storage with reuse on repeat requests (avoids redundant ingestion).
- A working hybrid retrieval strategy (dense + lexical) via `EnsembleRetriever`.
- A configurable retriever factory supporting five strategies plus two optional wrappers.
- Cross-encoder reranking, implemented and validated in the evaluation scripts.
- Separate, quantitative retrieval-quality evaluation (Recall/Precision/F1) and generation-quality evaluation (Ragas metrics) against labeled data.
- Latency instrumentation in the live pipeline (retrieval, prompt build, generation, each timed and logged).

- Bilingual (English/Arabic) transcript support with language-aware chunk separators and an explicit answer-language instruction in the generation prompt.

Needs Improvement

- Reranking and multi-query retrieval are implemented but not exercised by the live `/ask` path — the production pipeline is not using the full retrieval-quality machinery that exists in the codebase.
- `build_mmr_retrievers`'s `lambda_mult` parameter is non-functional (hardcoded internally), so MMR's diversity/relevance trade-off cannot currently be tuned by callers.
- Hardcoded, OS-specific (Windows) absolute paths throughout (`DATASETS_DIR`, `CHUNKS_DIR`, and all evaluation-script database/dataset paths) prevent the project from running unmodified on another machine or OS.
- Global mutable state (`rag_pipeline_v1`, `video_id_current`) in the FastAPI backend limits the system to one active video context across all clients, with no per-session isolation.
- `GET /Youtube_url` performs significant side effects (ingestion, DB writes) — inconsistent with HTTP GET semantics and with typical caching/proxy assumptions.
- No exception handling in either FastAPI endpoint — all backend errors surface as generic `500` responses with no structured error payload.
- `Semantic_Meaning_Test.py`'s `main()` currently exercises only one of four available Ragas metrics on one of three dataset files, even though the code to run all four concurrently (`Evaluate()`) already exists in the same file.
- Duplicated `calc_recall_precision_f1_score` implementations across `Exact_retrieval_metrics.py` and `build_ragas_dataset.py` (identical logic, not shared via a common module).
- Dead/unused code: the module-level `config` dict in `Pipeline/rag_pipeline.py` (instantiates an unused local `OllamaLLM`/`OllamaEmbeddings` pair at import time), the unused `PromptTemplate` import in `Exact_retrieval_metrics.py`, and the unused `chunk_documents_by_tokens` function.

Recommended Production Changes

- Replace global mutable pipeline state with per-session or per-request state (e.g. a keyed in-memory cache, or a real session/database layer) to support concurrent multi-user usage.
- Add structured error handling and appropriate HTTP status codes to both endpoints.
- Externalize all hardcoded paths and model names into environment variables or a configuration file, validated at startup.
- Decide whether reranking and/or multi-query retrieval should be part of the production `/ask` path, and wire them in (or explicitly document why they are evaluation-only).
- Fix the `lambda_mult` pass-through in `build_mmr_retrievers`.
- Add authentication and rate limiting to the API.
- Replace `print()`-based latency logging with structured logging/metrics export suitable for monitoring.
- Automate the currently-manual evaluation-dataset-generation step so the full evaluation pipeline can run without a human-mediated LLM call in the middle.
- Consolidate the duplicated metric-calculation function into a single shared module used by both evaluation scripts.

This is documentation of the existing implementation and its readiness characteristics — no refactoring of the underlying code was performed as part of producing this document.